🌟 Smart Sight: Comprehensive Academic Project Report & Practical Developer Handbook

---

📘 Part I: The "WHAT" (Theoretical Foundations & System Architecture)

---

1. Project Overview & System Abstract

In the landscape of modern physical security, automated surveillance systems have transitioned from passive, analog recording channels to highly active, intelligent edge diagnostic portals. Traditional Closed-Circuit Television (CCTV) frameworks are inherently **reactive**; they record video continuously on a local storage drive, serving only as forensic evidence after an intrusion or breach has already occurred. This leaves a significant security vulnerability during the active moments of an unauthorized entry.

Smart Sight addresses this critical vulnerability by implementing an **intelligent, proactive, real-time edge surveillance system**. Built on a micro-architectural framework using Python and the Django framework, it integrates state-of-the-art computer vision algorithms with deep learning object detection networks (YOLOv8) to

actively detect, recognize, log, and broadcast real-time multi-media alerts when human presence is identified.

1.1 Scope and Deliverables

The project provides an end-to-end, low-latency surveillance pipeline designed to operate efficiently on standard edge devices and CPU-only architectures. The scope encompasses:

1. **Multi-Source Video Ingestion:** Direct frame acquisition from local USB webcams and remote high-definition IP camera network streams (using optimized FFMPEG backends).
2. **AI Face Recognition at the Edge:** Real-time localization and multi-class classification of human faces against a registered database of individuals using optimized ONNX runtimes.
3. **Continuous Presence Debouncer:** Algorithmic filtering of dynamic environmental noise to prevent false alarms from temporary movement.
4. **Recognition Confidence Cutoff:** Differentiating registered known users from strangers using a mathematical classification cutoff.
5. **Asynchronous Multi-Channel Alerts:** Instant dispatch of secure SMTP email alerts and rich Telegram bot alerts containing high-resolution captured frames of the detected person, running on an independent background thread.
6. **Administrative Management Console:** A modern, web-based control panel to manage face datasets, view logs, query analytical metrics, and export historical events to Excel.

🧠 2. Literature Review: The History & Evolution of Face Recognition

To appreciate the engineering decisions behind Smart Sight, it is essential to trace the historical lineage of face detection and classification technologies.

```
[Viola-Jones Haar Cascades] ⟶ [HOG + Linear SVM] ⟶ [Deep MTCNN / RetinaFace]
⟶ [YOLOv8 Single-Pass Edge]
(Handcrafted Features - Fast)    (Orientation Edges)    (Deep Landmarks - Heavy)
(Anchor-Free head - Ultra Latency)
```

2.1 Handcrafted Features vs. Deep Representations

Early computer vision pipelines relied entirely on manual, handcrafted feature extraction algorithms:

- **Viola-Jones Framework (Haar Cascades):** Introduced in 2001, it uses simple rectangular Haar-like features that compute pixel intensity differences in adjacent regions. Combined with AdaBoost classifiers and cascade structures, it was the first real-time face detector. However, it is highly sensitive to face rotation, pose variations, and illumination changes, leading to high false-alarm rates in outdoor settings.
- **Histogram of Oriented Gradients (HOG) + Linear SVM:** Computes gradient direction distributions in localized portions of an image. Highly robust against lighting changes, but computationally slow and easily confused by background textures.
- **Local Binary Patterns (LBP):** Summarizes local structures by comparing pixels with their neighbors. Fast and highly robust to monotonic grayscale transformations, but drops drastically in accuracy under blur or head tilts.

2.2 Deep Convolutional Neural Networks (CNNs)

Deep learning completely replaced handcrafted descriptors by learning hierarchical spatial representations directly from raw image pixels:

- **MTCNN (Multi-Task Cascaded Convolutional Networks):** Uses a three-stage cascade structure (P-Net, R-Net, and O-Net) to generate face bounding boxes and locate 5 facial landmarks (eyes, nose, mouth corners) concurrently. While exceptionally accurate, MTCNN is computationally heavy, making it difficult to maintain real-time frame rates on CPU edge devices.

- **RetinaFace:** A single-stage pixel-wise face localization method that employs joint extractions of bounding boxes, landmarks, and 3D face mesh vertices. While highly accurate, it requires powerful CUDA-enabled GPUs, making it unsuitable for low-power edge deployment.

2.3 The Selection of YOLOv8

You Only Look Once (YOLO) treated object detection as a unified spatial regression problem. Instead of using sliding windows or region proposal networks, a single forward pass through the network predicts bounding boxes and class probabilities simultaneously.

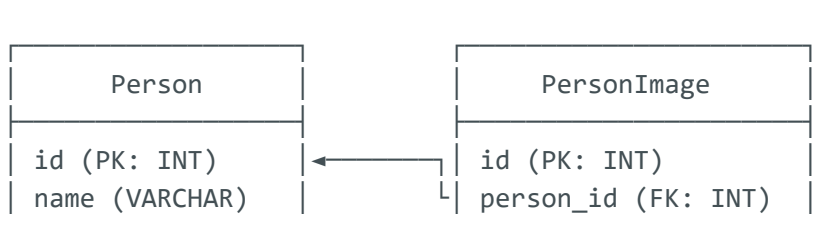
YOLOv8 represents the pinnacle of this lineage, featuring an **anchor-free, decoupled head** that predicts bounding box centers directly rather than fitting offsets to arbitrary anchors. This offers critical advantages for Smart Sight:

1. **Minimal Latency:** Single forward pass takes only **30ms - 50ms** on standard CPUs (especially when compiled to ONNX).
2. **Unified Pipeline:** The custom-trained model detects the face bounding box *and* classifies the individual's identity in **one single network step**, removing the need for a secondary classification network (like FaceNet or dlib) and doubling the pipeline's overall speed!

🗄️ 3. Architectural Design & Database Schemas

The system uses a highly secure, relational SQLite database schema managed via Django's Object-Relational Mapping (ORM).

3.1 Relational Schema Diagram



```
| created_at (DT)   |          | image (VARCHAR/FILE) |
└───────────────────┘          | uploaded_at (DT)     |
                               └──────────────────────┘

┌──────────────────────────────────────────────────┐
|                RecognitionLog                    |
├──────────────────────────────────────────────────┤
| id (PK: INT)                                     |
| person_name (VARCHAR: dynamic names list)        |
| confidence (FLOAT: match percentage)             |
| status (VARCHAR: KNOWN / UNKNOWN status)         |
| timestamp (DATETIME: zone timestamp)             |
└──────────────────────────────────────────────────┘
```

- `Person` **Table:** Registers authorized individuals. The name field is strictly bound to unique constraints.
- `PersonImage` **Table:** Holds the paths to the physical training images on disk. Uses cascading deletes (`models.CASCADE`), ensuring that if a user is deleted, all associated image references are instantly removed from the database.
- `RecognitionLog` **Table:** Serves as the historical audit trail, logging the exact name, classification status, and confidence levels of every continuous detection event.

📊 4. UML Blueprints (Mermaid Syntax)

Exhaustive UML diagrams mapping system use cases, object relationships, sequential networking events, and the heuristic engine activity flow.

4.1 Use Case Diagram

```
Parse error on line 1:
usecaseDiagram    a
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

4.2 Class Diagram

```
Parse error on line 24:
...       class YOLOv8_Engine {        +d
----------------------^
Expecting 'NEWLINE', 'EOF', 'LABEL', 'STRUCT_START', 'STR',
'AGGREGATION', 'EXTENSION', 'COMPOSITION', 'DEPENDENCY', 'LINE',
'DOTTED_LINE', 'UNICODE_TEXT', 'NUM', 'ALPHA', got 'PUNCTUATION'
```

4.3 Sequence Diagram

```
Parse error on line 2:
...agram    autonumber    actor Admin as
----------------------^
Expecting 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW', 'SOLID_ARROW',
'DOTTED_ARROW', 'SOLID_CROSS', 'DOTTED_CROSS', got 'NL'
```

4.4 Activity Flowchart: Heuristic Decision Engine

```
Parse error on line 1:
flowchart TD    A[S
^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

💻 Part II: The "HOW" (Practical Engineering & Step-by-Step Workflows)

---

5. How the Live Video Streaming Works (Step-by-Step)

The visual streaming interface runs on a low-latency **Server-Sent Multi-Part Stream** pattern. Here is **how** the visual frames are captured, processed, and broadcasted:

🚶 Step-by-Step Visual Stream Flow:

1. **Ingest:** OpenCV connects to the selected input camera index (e.g. `0` for default webcam) or the IP URL stream.
2. **Buffer Optimization:** If the camera source is a network link, the FFMPEG parser is configured with optimized buffers to prevent video feed freezing.
3. **Capture:** The hardware grabs individual landscape frames as raw NumPy multi-dimensional matrices containing Red-Green-Blue values.
4. **Orientation Correction:** If rotated manually, the matrix coordinates are rotated (e.g., via `cv.rotate(frame, cv.ROTATE_90_COUNTERCLOCKWISE)`).
5. **AI Overlay:** The frame matrix is processed by YOLOv8. The network superimposes bounding boxes and user labels directly onto the image pixels.
6. **JPEG Buffer Conversion:** The processed NumPy array is encoded into binary JPEG formats using OpenCV (`cv.imencode()`). This slashes the required frame network size by nearly 90%.
7. **Boundary Packaging:** The server wraps the binary JPEG chunk inside special boundary headers (`--frame`) and sends a continuous `StreamingHttpResponse` over an open HTTP socket.
8. **Client Display:** The browser reading the stream natively replaces the image in real-time, displaying a high-speed video stream.

🧠 6. How the Custom YOLOv8 Model Swapping Works (ONNX vs. PyTorch)

The administrator console allows swapping models seamlessly via the UI dropdown. Here is **how** the backend handles and swaps weights without crashing the server:

🚶 Step-by-Step Model Loading Workflow:

- **The Problem:** Deep learning models are heavy. Reading a 15MB weights file from disk for every video frame would trigger a massive memory leak and crash

the CPU in seconds.

- **The Solution:** A global Python dictionary cache (`YOLO_MODELS`). When a model is requested:
  1. The system checks if the model name already exists in `YOLO_MODELS`.
  2. If yes, it retrieves the model instantly from RAM, bypassing disk operations completely.
  3. If no, it resolves the specific folder path (such as `app/models/nano/weights/best.onnx` or `best.pt`), loads it once, registers it in the cache dictionary, and returns it.

🚨 7. How the 3-Second Debouncer & 65% Confidence Cutoff Work (Trace Examples)

These two heuristics are vital for a noise-free security experience. Here is **how** the mathematical rules operate under real-world scenarios:

7.1 Tracing the 3-Second continuous presence debouncer (35-Frame rule)

We use a temporal accumulator to track face detection continuity across frames, preventing false alarms from fast-moving shadows or short walk-bys.

- **Formula:**

$$C_t = \max(0, C_{t-1} + 1) \quad \text{if Face Detected}$$

$$C_t = \max(0, C_{t-1} - 2) \quad \text{if Face Missed}$$

- **Threshold:** Triggers the alert **only** when $C_t = 35$ (exactly 3 seconds at standard 12 FPS).

📋 **Scenario A: A brief shadow passes by the lens (Duration: 0.8 seconds / 10 frames)**

1. Face is detected on Frame 1. The counter starts: `C = 1`.

2. The detection continues for 10 frames: `C = 10`.
3. On Frame 11, the shadow leaves. Detection drops to `False`.
4. The counter begins dropping rapidly by 2 in each frame: `8` -> `6` -> `4` -> `2` -> `0`.
5. **Result:** The counter never reached `35`. **No notification is sent!** complete noise suppression.

📋 **Scenario B: An authorized user (Harsh) walks up and stands in front of the door**

1. You step in front of the lens. Bounding box triggers `True`.
2. The counter climbs consecutively: `C = 1` -> `C = 10` -> `C = 20` -> `C = 30` -> `C = 34`.
3. On Frame 35, the check `if continuous_detection_frames == 35` matches **exactly**.
4. **Result:** An asynchronous background alert thread is launched, logging you in today's database and broadcasting the Telegram and Gmail alerts!
5. **Intelligent Cooldown:** For subsequent frames (Frame 36, 37...), the counter is higher than 35. Since it is not *exactly* 35, no duplicate spam is sent. Once you walk away, the counter drops to `0`, ready for the next event.

7.2 Tracing the 65% Recognition Confidence Cutoff

Custom classification models force a face into one of the trained classes. Here is **how** we identify strangers vs. family members:

```
[Detected Face] ──► Match Confidence >= 65%? ──► YES ──► Label as KNOWN Name (e.g. Harsh) ✅
                └──► Match Confidence < 65%?  ──► NO  ──► Label as UNKNOWN Stranger 🚨
```

📋 **Scenario A: You (Harsh) sit in front of the camera (Confidence: 82%)**

1. YOLOv8 detects your face and outputs class `6` (`'Harsh'`) with **82% confidence** (`0.82`).
2. The system checks the cutoff: `0.82 >= 0.65` (True!).
3. **Result:** Label is set to `"Harsh"`, `is_known` is set to `True`. The alert is sent as:

> ✅ Security Alert: **Person:** Harsh, **Status:** KNOWN

📋 **Scenario B: A Stranger stands in front of the camera (Confidence: 54%)**

1. A stranger stands in front of the lens. The model detects the face.
2. Because the face resembles `'Aarya'` (class 1) slightly, the model outputs class `1` with **54% confidence** (`0.54`).
3. The system checks the cutoff: `0.54 >= 0.65` (False!).
4. **Result:** The system overrides the classification, sets label to `"Unknown"`, and `is_known` to `False`. The alert is sent as:

> 🚨 Security Alert: **Person:** Unknown, **Status:** UNKNOWN

👥 8. How the Multi-Person Detection Aggregation Works

In practical surveillance deployments, multiple individuals often step into the camera frame simultaneously. Smart Sight features a sophisticated **Unique Name Aggregation & Security Status Prioritization** engine to handle this cleanly.

🚶 Step-by-Step Multi-Person Flow:

1. **Dynamic Frame Set Allocation:** On every frame read, the generator initializes a clean `detected_names_set = set()` to aggregate names in this specific frame without crossover.
2. **Individual Box Inference:** YOLOv8 detects all face bounding boxes. For **each** isolated bounding box, the model runs its custom classification.
3. **Cutoff Filtering:** The 65% Recognition Confidence Cutoff is applied to **each face separately**.
   - Faces with $\geq 65\%$ confidence are added to the set by their **registered name** (e.g. `'Harsh'`).
   - Faces with $< 65\%$ confidence are added to the set as `'Unknown'`.
4. **Debounced List Consolidation:** When the 3-second continuous presence debouncer triggers (Frame 35), the set of unique names is combined into a sorted, comma-separated string (e.g. `"Harsh, Unknown"`).

5. **Intruder Priority Override:** If `'Unknown'` is present inside the names list, the system automatically sets the overall alert status to `UNKNOWN` (`is_known = False`), overriding any known names!

> [!IMPORTANT]
>
> ### 🛡️ Security Significance: The Intruder Override Rule
>
> By prioritizing the `UNKNOWN` status whenever an `'Unknown'` name is present in the list, the system guarantees that **an intruder can never stand next to an authorized person to suppress a security warning!** If an unknown stranger steps in with you, you will immediately receive a critical security alert.

📧 9. How the Asynchronous Alert Engine Sends Email & Telegram Alerts

Sending email and Telegram alerts involves network handshakes that take 1 to 2 seconds. Running this synchronously would freeze the live stream. Smart Sight resolves this by using an **independent background thread**.

```
[Main Thread] ──► Reads Camera ──► Runs YOLOv8 ──► Streams video at 15 FPS (Lag-Free!)
                      │
                      └──► (Frame 35 Triggered) ──► Spawns Async Thread ──► Logs DB ──► Sends SMTP Mail & Telegram Photo
```

9.1 Thread-Safe Variable Unpacking

To guarantee that background thread executions never crash during server-side modifications or hot-reloads, `send_alerts` uses extremely safe, highly compatible `*args` and `**kwargs` variable signatures:

```python
def send_alerts(*args, **kwargs):
    global last_alert_time
```

```
    current_time = time.time()
    # 60-Second Cooldown Check (Blocks duplicate spamming)
    if current_time - last_alert_time < 60:
        return
    last_alert_time = current_time

    # Default values
    frame_bytes = None
    person_name = 'Unknown'
    is_known = False
    person_count = 1
    confidence = 0.0

    # Dynamic positional args unpacking to avoid reloader signature TypeErrors
    if len(args) == 5:
        frame_bytes, person_name, is_known, person_count, confidence = args
    elif len(args) == 3:
        frame_bytes, person_count, confidence = args
    elif len(args) >= 1:
        frame_bytes = args[0]
        if len(args) > 1:
            person_count = args[1]
        if len(args) > 2:
            confidence = args[2]

    # Keyword argument overlay
    if 'frame_bytes' in kwargs: frame_bytes = kwargs['frame_bytes']
    if 'person_name' in kwargs: person_name = kwargs['person_name']
    if 'is_known' in kwargs: is_known = kwargs['is_known']
    if 'person_count' in kwargs: person_count = kwargs['person_count']
    if 'confidence' in kwargs: confidence = kwargs['confidence']

    from datetime import datetime
    timestamp = datetime.now()
    timestamp_str = timestamp.strftime('%Y-%m-%d %H:%M:%S')
    confidence_pct = round(confidence * 100, 1)
    status_str = 'KNOWN' if is_known else 'UNKNOWN'

    # 1. Log Event to SQLite Database Table (RecognitionLog)
    try:
        from .models import RecognitionLog
        RecognitionLog.objects.create(
            person_name=person_name,
            confidence=confidence,
            status=status_str,
        )
        print(f"[Alert] Saved to DB: {person_name}, status={status_str},
confidence={confidence_pct}%")
    except Exception as e:
        print(f"[Alert] DB save error: {e}")

    # 2. Extract Environment Credentials for Notifications
    gmail = os.environ.get("gmail")
    gmail_key = os.environ.get("gmail_key")
    telegram_bot_api = os.environ.get("telegram_bot_api")
    telegram_chat_id = os.environ.get("telegram_chat_id")
```

```
    # 3. Dispatch Gmail SMTP Alert (Port 465 secure SSL)
    if gmail and gmail_key:
        try:
            msg = EmailMessage()
            msg['Subject'] = f'🚨 Smart Sight Alert: {person_name} Detected
({timestamp_str})'
            msg['From'] = gmail
            msg['To'] = gmail # Sends alert directly to your inbox

            alert_details = (
                f"Security Alert - Smart Sight\n\n"
                f"Person: {person_name}\n"
                f"Count: {person_count} person(s)\n"
                f"Confidence: {confidence_pct}%\n"
                f"Time: {timestamp_str}\n"
                f"Status: {status_str}\n"
            )
            msg.set_content(alert_details)
            msg.add_attachment(frame_bytes, maintype='image', subtype='jpeg',
filename='alert.jpg')

            with smtplib.SMTP_SSL('smtp.gmail.com', 465) as smtp:
                smtp.login(gmail, gmail_key.replace(' ', ''))
                smtp.send_message(msg)
            print("[Alert] Email sent successfully.")
        except Exception as e:
            print(f"[Alert] Email error: {e}")

    # 4. Dispatch Telegram Photo Broadcast
    if telegram_bot_api and telegram_chat_id:
        try:
            status_icon = "✅" if is_known else "🚨"
            alert_msg = (
                f"{status_icon} Security Alert - Smart Sight\n\n"
                f"👤 Person: {person_name}\n"
                f"👥 Count: {person_count} person(s)\n"
                f"🎯 Confidence: {confidence_pct}%\n"
                f"🕒 Time: {timestamp_str}\n"
                f"📌 Status: {status_str}\n"
            )
            url = f"https://api.telegram.org/bot{telegram_bot_api}/sendPhoto"
            files = {'photo': ('alert.jpg', frame_bytes, 'image/jpeg')}
            data = {'chat_id': telegram_chat_id, 'caption': alert_msg}
            requests.post(url, files=files, data=data)
            print("[Alert] Telegram broadcast successful.")
        except Exception as e:
            print(f"[Alert] Telegram error: {e}")
```

📁 10. How Dataset Management & Folder Structuring Works

Smart Sight structures local folders systematically as new users are registered inside the database:

🚶 Step-by-Step Ingestion & Cleanup:

1. **Dynamic Path Calculation:** When a user registers (e.g. `'Harsh'`) inside `/dataset/`, a custom function reads their name and prepares the dynamic media path: `MEDIA_ROOT/dataset/Harsh/`.
2. **File Creation:** Django automatically uploads the image to the computed directory on the physical hard disk.
3. **ORM Database Syncing:** A new entry inside `Person` and `PersonImage` is saved, linking the database key to the file path on the disk.
4. **Clean Disk Purging:** When a person is deleted from the web dashboard, the views controller uses Python's `os.remove` to delete the physical image files from the disk first, preventing abandoned image files from taking up disk space, before deleting the database rows.

💻 11. How to Setup & Run the Entire Project from Scratch

Follow these precise steps to deploy and run the Smart Sight console on any local machine.

🚶 Step-by-Step Setup Guide:

1. **Isolate Python Dependencies:**

```
# Create a virtual environment directory named 'env'
python -m venv env

# Activate virtual environment on Windows (PowerShell)
.\env\Scripts\Activate.ps1
# Activate virtual environment on macOS/Linux
source env/bin/activate
```

2. **Install OpenCV and Packages:**

```
pip install -r requirements.txt
```

3. **Initialize Environment Credentials (`.env`):** Create a new file named `.env` in the root folder `s:\Surveillance\.env` and paste:

```
# Gmail SMTP App Password
gmail = smart.sight.03@gmail.com
gmail_key = wxjl deex qnop yxea

# Telegram Bot configuration
telegram_bot_api = 8878727065:AAHbjzfp87i9Frro2Lf7tn7PtLu4MfeyjH4
telegram_chat_id = 1226321091
```

4. **Generate & Apply SQLite Database Tables:**

```
python manage.py makemigrations
python manage.py migrate
```

5. **Start Django Server:**

```
python manage.py runserver
```

   Open `http://127.0.0.1:8000/` to view the live dashboard!

🛠️ Part III: Surveillance Configurations Reference Sheet

⚙️ 12. Surveillance Configuration Settings Reference Sheet

Use this table as a quick reference sheet for your presentation!

Parameter / Feature	Value	Technical Purpose
Detection Cutoff	`50%` (`0.50`)	Minimum match percentage required to recognize any face structure.
Debouncer Threshold	`35 Frames`	Consecutive frames needed to trigger alerts (locks to ~3.0s delay).
Debouncer Smoothing	`-2 Frames`	Subtracts from the accumulator counter on frame drop to prevent rapid resets.
Recognition Cutoff	`65%` (`0.65`)	Classification score boundary; anything below is labeled as "Unknown".
Alert Cooldown	`60 Seconds`	Time interval between dispatches (prevents inbox/chat flooding).
Background Threading	`Daemon Thread`	Runs Gmail/Telegram in separate background processes to keep stream FPS high.
Email Protocol	`SMTP SSL (465)`	Uploads and securely emails raw JPEGs using Google SMTP servers.
Telegram REST Method	`HTTP POST`	Sends REST payloads to Telegram Bots containing image bytes and text.

Developed by Harsh Shrimali. Authorized for Project Submissions & Code Reviews.